

Dependency Preservation in Database Decomposition

1 Dependency Preservation

Definition: Dependency Preservation

When we decompose a relation schema R into smaller relations $R_1, R_2, \dots, R_n$, we want to ensure that **all functional dependencies** from the original schema F are still enforceable without reconstructing R through a join.

Formally:

- Let F^+ = closure of all functional dependencies F .
- Let F_i = restriction of F^+ to the attributes in R_i .
- The decomposition is **dependency preserving** if:

$$(F_1 \cup F_2 \cup \dots \cup F_n)^+ = F^+$$

This means that every dependency in F can be checked by enforcing dependencies only on the decomposed relations.

Why is Dependency Preservation Important?

If the decomposition is **not** dependency preserving:

- Some dependencies will only be verifiable by **joining** decomposed relations.
- Joins are **expensive** operations — they take more time and resources.
- It complicates **data integrity enforcement** after updates.

Dependency preservation ensures integrity can be checked efficiently on individual tables.

Example of the Problem

Suppose:

$$R = (A, B, C), \quad F = \{A \rightarrow B, B \rightarrow C\}$$

Decompose into:

$$R_1 = (A, B), \quad R_2 = (A, C)$$

Here:

- $A \rightarrow B$ can be checked in R_1 .
- But $B \rightarrow C$ cannot be checked in R_1 or R_2 individually.
- To check $B \rightarrow C$, we must join R_1 and R_2 .

Thus, the decomposition is **not** dependency preserving.

Algorithm for Testing Dependency Preservation

1. Compute F^+ — closure of all dependencies.

2. For each decomposed relation R_i :

$$F_i = \text{restriction of } F^+ \text{ to attributes of } R_i$$

3. Form:

$$F' = F_1 \cup F_2 \cup \dots \cup F_n$$

4. Compute F'^+ — closure of dependencies in F' .

5. If $F'^+ = F^+$, the decomposition is **dependency preserving**.

Simpler Practical Check

Instead of computing full closures:

- For each dependency $X \rightarrow Y$ in F , check if all attributes in X and Y are in at least one R_i .
- If true for all dependencies, decomposition is dependency preserving.

2 Worked Examples

Example 1: Dependency Preserving

$$R = (A, B, C), \quad F = \{A \rightarrow B, B \rightarrow C\}$$

Decompose into:

$$R_1 = (A, B), \quad R_2 = (B, C)$$

Check:

- $A \rightarrow B$ is in R_1 .
- $B \rightarrow C$ is in R_2 .

Since each dependency is contained in some relation, the decomposition is **dependency preserving**.

Example 2: Not Dependency Preserving

$$R = (A, B, C), \quad F = \{A \rightarrow B, B \rightarrow C\}$$

Decompose into:

$$R_1 = (A, B), \quad R_2 = (A, C)$$

Check:

- $A \rightarrow B$ is in R_1 — OK.
- $B \rightarrow C$ is not in R_1 or R_2 — FAIL.

To check $B \rightarrow C$, we must join R_1 and R_2 . Hence, **not dependency preserving**.

Example 3: Larger Schema

$$R = (A, B, C, D), \quad F = \{A \rightarrow B, BC \rightarrow D, D \rightarrow A\}$$

Decompose into:

$$R_1 = (A, B), \quad R_2 = (B, C, D)$$

Check:

- $A \rightarrow B$ is in R_1 — OK.
- $BC \rightarrow D$ is in R_2 — OK.
- $D \rightarrow A$ is not fully in R_1 or R_2 — FAIL.

Thus, this decomposition is **not dependency preserving**.

Example 4: Lossless but Not Dependency-Preserving Decomposition

Given:

$$R = (A, B, C, D, E), \quad F = \{A \rightarrow BC, CD \rightarrow E, B \rightarrow D, E \rightarrow A\}$$

Decompose into:

$$R_1 = (A, B, C), \quad R_2 = (A, D, E)$$

Check lossless:

- Common attributes: $R_1 \cap R_2 = \{A\}$.
- Since $A \rightarrow BC$ in F , $A \rightarrow (R_1 - R_2)$ holds.
- Therefore, the decomposition is **lossless**.

Check dependency preservation:

- $A \rightarrow BC$ is in R_1 — OK.
- $E \rightarrow A$ is in R_2 — OK.
- $B \rightarrow D$ spans both R_1 and R_2 — FAIL.
- $CD \rightarrow E$ spans both R_1 and R_2 — FAIL.

Thus, this decomposition is **not dependency preserving**.